

DILIGENT SOFTWARE SERVICES

Diligent Code Studio

Operator/User Manual

Version 0.6.10 | Windows x64 Release Package

Document purpose

This manual provides practical user and operator guidance for installing, launching, configuring, and maintaining Diligent Code Studio v0.6.10. It is written for end users, support personnel, and administrators who need a clean operating reference.

Prepared: May 27, 2026

Prepared for the v0.6.10 installer package provided in this project workspace.

1. Document Control

Item	Details
Product	Diligent Code Studio
Product version	0.6.10
Document version	1.0
Release platform covered	Windows x64
Primary audience	Operators, end users, support technicians, and software administrators
Prepared date	May 27, 2026

1.1 Revision History

Version	Date	Description
1.0	May 27, 2026	Initial operator/user manual for Diligent Code Studio v0.6.10.

1.2 Scope

This manual covers the Windows x64 v0.6.10 package and the major user-facing operating procedures: installation, first launch, dependency setup, workspace use, AI assistant use, settings, troubleshooting, and maintenance. The uploaded release package contained Windows installers only, so Linux and Debian installation steps are intentionally treated as future release guidance rather than final installation instructions in this edition.

2. Manual Contents

- 1. Document Control
- 2. Manual Contents
- 3. Product Overview
- 4. Safety, Security, and Operating Notes
- 5. System Requirements
- 6. Release Package Contents
- 7. Installation and Removal
- 8. First Launch and Initial Setup
- 9. User Interface Overview
- 10. Daily Operating Procedures
- 11. Workspace and File Management
- 12. Editor Operations
- 13. AI Assistant Operations
- 14. Dependency Setup Page
- 15. Settings and Interface Preferences
- 16. Operator Administration
- 17. Troubleshooting Guide
- 18. Backup, Upgrade, and Recovery

- 19. Support Information Checklist
- 20. Appendices

3. Product Overview

Diligent Code Studio is a desktop code editor and development workspace intended to make project work easier for users who want a clean editor, organized workspace navigation, dependency checks, and local AI assistance. Version 0.6.10 focuses on a practical operator experience: easier release packaging, an AI assistant available from the editor window by default, improved workspace navigation, a setup page for required tools, and settings that allow the interface to be simplified.

3.1 Primary Use Cases

- Open and work in local software project folders.
- Edit project files and review code from a single application window.
- Use the integrated AI assistant for explanations, code review, troubleshooting, and generation assistance.
- Check required development dependencies from a centralized setup page.
- Adjust the workspace menu and interface size for normal or compact use.
- Package and test projects without requiring the user to manually remember every supporting tool.

3.2 Intended Users

- Software developers who want a lightweight branded editor workflow.
- Operators who maintain Diligent Software Services projects.
- New users who need guided setup for common developer dependencies.
- Support technicians who need a consistent troubleshooting checklist.
- Administrators who need to install or remove the application on Windows systems.

4. Safety, Security, and Operating Notes

Operator caution

Diligent Code Studio edits real project files. Always keep a backup or use source control before making major changes. AI-generated code should be reviewed and tested before it is committed, packaged, or released.

- Back up important projects before running large code changes, dependency installs, build scripts, or automated refactoring steps.
- Do not paste passwords, API keys, private certificates, private SSH keys, customer data, or other secrets into the AI assistant unless you fully understand where that information is processed.
- Prefer a local AI provider such as Ollama when working with sensitive projects, and still avoid sending secrets into prompts.
- Dependency installation may require administrator rights. Only install dependencies from trusted sources.

- Treat build outputs, installers, scripts, and generated packages as software releases that require testing before distribution.
- When the app reports that a dependency is missing, verify from both the setup page and a terminal before assuming the tool is truly absent.

5. System Requirements

Requirement	Recommended
Operating system	Windows 10 or Windows 11, 64-bit. The package supplied for this manual is a Windows x64 release.
Processor	Modern 64-bit Intel or AMD processor.
Memory	8 GB RAM minimum; 16 GB RAM recommended for larger projects and AI-assisted workflows.
Disk space	At least 500 MB free for the application plus additional space for projects, dependencies, logs, AI models, and build outputs.
Network	Internet access recommended for downloading dependencies, updates, AI models, and project packages. Local-only use is possible after setup when using local tools.
Permissions	Standard user rights for daily use. Administrator rights may be required for some dependency installers.
Optional AI	Ollama installed and running locally when using local AI assistant features.

5.1 Common Optional Dependencies

The exact dependency list can change as the product evolves. Version 0.6.10 includes a setup/dependency workflow so users do not have to remember every prerequisite manually. Common tools that may be detected or installed include:

- Git, for source control and repository operations.
- Node.js and npm, for JavaScript, TypeScript, Electron, Tauri, web, and package workflows.
- Python, for scripting, automation, and project tooling.
- Ollama, for local AI model support.
- Visual Studio C++ Build Tools, for native Windows build requirements used by some packages and development workflows.

6. Release Package Contents

The v0.6.10 ZIP package inspected for this manual contained two Windows installer options. Either installer can be distributed to users, but the recommended installer depends on the operating environment.

File	Purpose	Recommended use
msi/Diligent Code Studio_0.6.10_x64_en-US.msi	Windows Installer package built for x64 systems.	Best for managed/admin deployment, scripted installation, logging, and enterprise-style installs.
nsis/Diligent Code Studio_0.6.10_x64-setup.exe	Standard setup executable.	Best for normal end users who expect a click-through setup experience.

6.1 File Integrity Hashes

Use these hashes to verify that the downloaded files match the package used for this manual. When posting public downloads, publish a checksums.txt file beside the installers.

Artifact	SHA-256
diligent-code-studio-windows-0.6.10.zip	cbacde0a6e728a248c9211469a1a7f6376724680230617e85050e6d5ff01ecee
Diligent Code Studio_0.6.10_x64_en-US.msi	7d8b99ddcfba6552af10561588de1a48ac446a219eaa7e49442fd06ed931a44e
Diligent Code Studio_0.6.10_x64-setup.exe	d8b99f99d3f68e86685c5688576a29de9870613abad754b3597a0abe087fee66

7. Installation and Removal

7.1 Choose the Correct Installer

- For most users: run the NSIS setup EXE from the nsis folder.
- For administrators or scripted deployments: use the MSI from the msi folder.
- Do not install both installers over each other unless testing upgrade behavior. Pick one standard installer method for the release.

7.2 Standard User Installation Using the Setup EXE

1. Download the official Diligent Code Studio v0.6.10 Windows ZIP package.
2. Right-click the ZIP file, choose Extract All, and extract it to a known folder such as Downloads or Desktop.
3. Open the nsis folder.
4. Double-click Diligent Code Studio_0.6.10_x64-setup.exe.
5. If Windows SmartScreen appears, verify the source of the download before continuing.
6. Follow the setup prompts until installation completes.
7. Launch Diligent Code Studio from the Start menu or desktop shortcut if one was created.

SmartScreen note

Unsigned or newly published installers may trigger a Windows SmartScreen warning. This does not automatically mean the installer is malicious, but users should only continue when the file came from the official project release location and the checksum matches.

7.3 Administrator Installation Using the MSI

1. Extract the release ZIP package.
2. Open an elevated Command Prompt or PowerShell window if administrator installation is required.
3. Change to the folder containing the MSI.
4. Run the MSI normally or with logging as shown below.

```
msiexec /i "Diligent Code Studio_0.6.10_x64_en-US.msi" /L*v "%TEMP%\DiligentCodeStudio_Install.log"
```

7.4 Quiet MSI Installation for Managed Deployment

Administrators can use the standard Windows Installer silent install pattern when deploying the MSI through a management tool or script.

```
msiexec /i "Diligent Code Studio_0.6.10_x64_en-US.msi" /qn /norestart /L*v "%TEMP%\DiligentCodeStudio_Install.log"
```

7.5 Removal / Uninstall

1. Close Diligent Code Studio.
2. Open Windows Settings.
3. Go to Apps > Installed apps.
4. Search for Diligent Code Studio.
5. Choose Uninstall and follow the prompts.
6. If performing a full reset, remove user-specific configuration only after confirming that no settings, workspaces, or logs need to be preserved.

8. First Launch and Initial Setup

The first launch should be treated as a setup and verification session. The goal is to confirm the application opens correctly, the workspace location is available, and required developer dependencies are installed or marked as optional.

8.1 First Launch Checklist

- Launch Diligent Code Studio from the Start menu.
- Open or create a workspace folder for your project.
- Open the Setup or Dependencies page and run the dependency check.
- Install any required tools using the available installer buttons or the documented vendor installer.
- Refresh or re-run dependency detection after each install.
- Configure AI assistant settings, especially if using Ollama locally.
- Review Settings and choose normal or compact interface mode.
- Confirm the workspace menu order is correct, then drag-and-drop workspace buttons if needed.

8.2 Recommended First Test

1. Open a non-critical test project folder.
2. Create or open a simple text or code file.
3. Type a small change and save the file.
4. Open the AI assistant and ask it to explain the current file or a selected code section.
5. Open the Setup page and verify that expected tools are detected.
6. Close and reopen the application to confirm settings persist.

9. User Interface Overview

The interface is organized around a workspace-centered development flow. Exact labels may vary slightly as the product evolves, but the main operating areas are described below.

Area	Purpose
Workspace Menu / Navigation	Provides quick access to major app areas and project functions. In v0.6.10, workspace button order is intended to be managed by drag-and-drop rather than a large manual ordering section.
Editor Window	Main area for viewing and editing project files. The AI assistant is available from the editor window by default and can be hidden or turned off when not needed.
File / Project Area	Used to browse and open files in the active workspace.
AI Assistant Panel	Used for code explanations, troubleshooting, code generation, and project assistance.
Setup / Dependencies Page	Checks for required tools and provides install actions or guidance.
Settings	Controls interface preferences, workspace menu reset, AI behavior, and other user options.
Status / Output Areas	Shows status messages, logs, build output, terminal output, or dependency results where supported.

9.1 Normal Interface Mode

Normal mode is the best default for new users. It typically uses more spacing, larger visible labels, and clearer navigation. Use this mode when training users, writing documentation, recording demonstrations, or troubleshooting user reports.

9.2 Compact Interface Mode

Compact mode reduces interface size and visual spacing so more of the editor and workspace can fit on screen. This is useful on smaller monitors, laptops, or when the user wants less navigation clutter. Compact mode should not remove critical functionality; it should only make the interface tighter.

9.3 Workspace Menu Drag-and-Drop

Workspace menu ordering should be handled directly through drag-and-drop. A reset option may be available in Settings for restoring the default order. The reset button should remain in Settings rather than on the workspace menu itself, so the daily navigation area stays clean.

10. Daily Operating Procedures

10.1 Start of Work Procedure

1. Start Diligent Code Studio.
2. Open the correct workspace folder.
3. Verify the active project is the intended project before editing files.
4. Check source control status if the project uses Git.

5. Confirm AI assistant mode if you plan to use AI support.
6. Review any dependency or environment warnings before building or running project tasks.

10.2 Normal Shutdown Procedure

1. Save open files.
2. Complete or cancel running tasks, terminals, builds, or dependency operations.
3. Review source control status and commit changes when appropriate.
4. Close the workspace or exit Diligent Code Studio.
5. Back up or publish release artifacts only after testing them.

10.3 Safe Change Procedure

- Make one logical change at a time.
- Use source control or manual backups before broad refactoring.
- Run the project after important changes.
- Read AI suggestions before applying them.
- Avoid accepting large AI-generated rewrites without reviewing file paths, deleted logic, package versions, and error handling.

11. Workspace and File Management

11.1 Opening a Workspace

1. Choose the Open Workspace or Open Folder action.
2. Select the root folder of the project, not a random subfolder, unless you intentionally want a narrow workspace.
3. Allow the file tree or project view to load.
4. Confirm that expected files such as README, package files, source folders, or project files are visible.

11.2 Recommended Workspace Rules

- Keep projects in a dedicated folder such as C:\DiligentProjects or another known working location.
- Avoid editing directly inside Downloads, temporary extraction folders, or installer output folders.
- Do not use spaces or unusual characters in critical build paths when a project toolchain is sensitive to paths.
- Keep source files separate from release artifacts such as MSI, EXE, ZIP, AppImage, or tar.gz outputs.
- Use Git or another source control system for projects that will be released publicly.

11.3 File Editing Practices

- Open the file from the workspace tree or file list.
- Make edits in the editor pane.

- Save before switching branches, building, packaging, or closing the app.
- Use descriptive filenames and avoid overwriting installer or release assets.
- When testing generated code, create a backup copy or commit before replacing large files.

12. Editor Operations

12.1 Basic Editing

- Use the editor pane for normal text and code edits.
- Save frequently, especially before running build or packaging commands.
- Use search within files when changing repeated labels or version numbers.
- Check that version strings are consistent across package files, installer names, changelogs, release notes, and documentation.

12.2 Working with Release Version Numbers

When preparing a release, make sure the version number is consistent everywhere. For this manual, the correct working version is 0.6.10. Do not use old version labels such as 0.4.8 in release names, installer names, workflow artifacts, documentation, download instructions, or website copy.

Location	Check
Application package metadata	Version should show 0.6.10.
Installer filenames	Use v0.6.10 or 0.6.10 consistently.
GitHub Actions artifacts	Artifact names should include 0.6.10 when publishing this release.
GitHub Release title	Use Diligent Code Studio v0.6.10.
Website download page	Download buttons and install instructions should reference v0.6.10.
Manual and support docs	Document title, revision table, and support checklist should reference v0.6.10.

13. AI Assistant Operations

The AI assistant is intended to help with explanations, troubleshooting, code suggestions, and documentation. In the v0.6.10 direction, the assistant should be available from the editor window by default and should be easy to hide or disable when the user wants more editing space.

13.1 Common AI Assistant Tasks

- Explain a file or selected code block.
- Suggest a fix for an error message.
- Generate a starter function, UI section, script, or documentation section.
- Review code for obvious mistakes before testing.
- Summarize project changes before writing release notes.
- Help create install or troubleshooting steps for non-technical users.

13.2 Safe AI Workflow

1. Select only the code or error message needed for the question.
2. Ask a direct question with the expected outcome.
3. Review the answer before applying it.
4. Apply changes manually or in small patches.
5. Run the project or relevant tests.
6. Commit or document the change only after the project works.

13.3 Local AI with Ollama

When using local AI, make sure Ollama is installed, running, and has at least one model available. If the setup page says Ollama is missing even though it is installed, use the troubleshooting steps in Section 17.

```
ollama --version
ollama list
ollama pull llama3.2
ollama serve
```

14. Dependency Setup Page

The dependency setup page is intended to reduce setup confusion by listing important tools, showing whether each one is detected, and providing an install button or setup action for each dependency where practical.

14.1 How to Use the Setup Page

1. Open the Setup or Dependencies page from the workspace menu.
2. Run the dependency check or refresh action.
3. Review the status of each dependency.
4. Install missing required dependencies one at a time.
5. After each installation, close and reopen Diligent Code Studio or run the refresh action.
6. Confirm the dependency status changes from missing to installed or detected.
7. Document any dependency that still fails detection after manual verification.

14.2 Dependency Status Meaning

Status	Meaning	Operator Action
Installed / Detected	The tool was found by the application.	No action required. Optionally verify the version if troubleshooting.
Missing	The tool was not detected by the app.	Install the tool or verify it is available in PATH. Then refresh detection.
Optional	The tool is useful for some features but not required for all users.	Install only if your workflow needs it.
Install failed	The installer did not complete normally or returned a warning/exit code.	Read the detailed message. Some installers report no update available when the tool is already installed.

Unknown	The app could not determine status.	Verify manually from PowerShell or Command Prompt and report details if needed.
---------	-------------------------------------	---

14.3 Dependency Install Best Practices

- Install one dependency at a time so errors are easier to identify.
- Use administrator rights when the dependency installer requires system-level changes.
- Restart the app after installing tools that update PATH.
- Restart Windows if the dependency installer recommends it.
- Record exact error messages and exit codes for support.

15. Settings and Interface Preferences

15.1 Recommended Default Settings

Setting Area	Recommended Default
Interface mode	Normal mode for new users; compact mode for experienced users or smaller screens.
AI assistant visibility	Visible by default on the editor window, with the ability to hide or disable it.
Workspace menu order	User-controlled by drag-and-drop. Reset option available in Settings.
Dependency detection	Enable manual refresh or re-check after installations.
Theme / appearance	Use the clearest readable theme for the operator environment.

15.2 Resetting Workspace Menu Order

1. Open Settings.
2. Find the workspace menu reset option.
3. Confirm the reset if prompted.
4. Return to the workspace menu and verify that the default order is restored.
5. Use drag-and-drop to make any preferred adjustments.

16. Operator Administration

16.1 Release Preparation Checklist

- Confirm the version is 0.6.10 across package files, workflows, installer metadata, release notes, website copy, and documentation.
- Build the Windows package and verify the MSI and setup EXE launch successfully on a clean test system.
- Generate SHA-256 checksums for all release files.
- Upload release artifacts to the official GitHub Release or official download location.
- Publish a checksums.txt file.
- Update the download page and installation instructions.

- Install the release on a separate test machine before announcing it publicly.

16.2 Generating Checksums in PowerShell

Use PowerShell to generate a SHA-256 checksum for release files before publishing them.

```
Get-FileHash "Diligent Code Studio_0.6.10_x64-setup.exe" -Algorithm SHA256
```

```
Get-FileHash "Diligent Code Studio_0.6.10_x64_en-US.msi" -Algorithm SHA256
```

16.3 Recommended Release Folder Structure

```
DiligentCodeStudio-v0.6.10/
  windows/
    msi/
    nsis/
  checksums.txt
  release-notes.md
  manual/
    Diligent_Code_Studio_v0.6.10_Operator_User_Manual.pdf
```

17. Troubleshooting Guide

17.1 App Will Not Install

Symptom	Likely Cause	Recommended Action
Windows blocks the setup file	SmartScreen or file downloaded from an unrecognized location.	Verify official source and SHA-256 checksum. Right-click the file, choose Properties, and unblock only when trusted.
Installer requires admin rights	System-level installation path or dependency operation.	Run installer as administrator or use the MSI with an elevated command prompt.
MSI install fails	Windows Installer error or existing installation conflict.	Run with logging using /L*v, uninstall the older version if needed, and review the log.
Setup EXE fails silently	Installer blocked, permissions issue, or interrupted extraction.	Try the MSI installer, run from a local folder, and avoid network paths.

17.2 Ollama Is Installed but Shows Missing

This can happen when the app detection logic cannot see the same PATH or service state that a user shell can see. Use the following checks:

1. Open PowerShell and run `ollama --version`.
2. Run `ollama list` to confirm at least one model is available.
3. Start the Ollama service or run `ollama serve` if needed.
4. Restart Diligent Code Studio after installing Ollama because PATH changes may not be visible to an already-running app.
5. Open the Setup page and run the dependency refresh again.
6. If detection still fails but Ollama works from PowerShell, report the PowerShell output, install path, Windows version, and Diligent Code Studio version.

17.3 Visual Studio C++ Build Tools Reports No Upgrade Available

A package manager or installer may report that Visual Studio C++ Build Tools is already installed and that no newer version is available. That message is not always a true failure. It may simply mean the required package already exists.

- Refresh the dependency check after the installer finishes.
- Open Visual Studio Installer and verify C++ build tools are present if the setup page still reports missing.
- Record the exact exit code and message for support.
- If the message says no available upgrade found, treat it as an install-state warning rather than a confirmed missing dependency until manually verified.

17.4 Workspace Menu Cannot Be Reordered

- Confirm the current build includes drag-and-drop workspace menu support.
- Try dragging from the button body rather than only the icon or label.
- Check whether compact mode changes the drag handle area.
- Restart the application and try again.
- Use Settings to reset the menu order, then retry drag-and-drop.
- Report the screen size, interface mode, and exact menu item that will not move.

17.5 AI Assistant Does Not Respond

- Verify the selected AI provider is configured correctly.
- For local AI, confirm Ollama is running and the chosen model is installed.
- Restart the application after changing AI settings.
- Try a short test prompt before pasting large files or logs.
- Check whether network, firewall, antivirus, or model download issues are preventing connection.
- Record the exact prompt, provider, model, and error message for support.

17.6 Dependency Installer Button Does Nothing

- Confirm Windows did not open a hidden User Account Control prompt behind another window.
- Run Diligent Code Studio as administrator only when necessary to start a system installer.
- Try installing the dependency manually from the vendor or official package manager.
- Restart the app and re-run detection after manual installation.
- Record dependency name, expected version, and any installer output.

18. Backup, Upgrade, and Recovery

18.1 Before Upgrading

- Close Diligent Code Studio.
- Commit or back up project changes.
- Save copies of custom settings, workspace lists, or configuration files if they are critical.

- Download the new installer from the official release location.
- Verify the checksum before installing.
- Install the new version and confirm the About or version display shows the expected version.

18.2 Recovery from a Bad Setting

1. Restart the application.
2. Open Settings if accessible.
3. Reset interface layout or workspace menu order.
4. Disable optional AI or dependency features if they are causing startup issues.
5. If the app cannot open, uninstall and reinstall the application, preserving project folders.
6. Only remove user configuration folders after backing up anything that may be needed.

18.3 Project File Recovery

- Use Git history when available.
- Use backup copies created before large AI-assisted rewrites.
- Check editor autosave or backup behavior if implemented in the current build.
- Avoid continuing to edit after accidental deletion until recovery options are checked.
- For important projects, store source code in a repository before using the editor for major changes.

19. Support Information Checklist

When reporting a problem, provide enough detail for the support person to reproduce the issue. Include screenshots when possible.

Information	Example / Notes
Product version	Diligent Code Studio v0.6.10.
Installer used	MSI or NSIS setup EXE.
Operating system	Windows 10 or Windows 11, edition, and build if known.
Install context	Standard user, administrator, managed deployment, or portable test environment.
Workspace path	Example: C:\DiligentProjects\ProjectName. Do not include sensitive customer paths if not needed.
Dependency status	List tools shown as missing and tools confirmed manually.
AI provider/model	Example: Ollama with llama3.2, if relevant.
Error text	Copy the exact error message and exit code.
Steps to reproduce	List the exact sequence of clicks or commands.
Expected result	What you expected to happen.
Actual result	What actually happened.

20. Appendices

20.1 Quick Reference Commands

Purpose	Command
Install MSI with log	<code>msiexec /i "Diligent Code Studio_0.6.10_x64_en-US.msi" /L*v "%TEMP%\DiligentCodeStudio_Install.log"</code>
Quiet MSI install	<code>msiexec /i "Diligent Code Studio_0.6.10_x64_en-US.msi" /qn /norestart /L*v "%TEMP%\DiligentCodeStudio_Install.log"</code>
Uninstall MSI by package	Use Windows Settings > Apps, or your software deployment tool.
Check a file hash	<code>Get-FileHash "filename.ext" -Algorithm SHA256</code>
Check Ollama version	<code>ollama --version</code>
List Ollama models	<code>ollama list</code>
Pull local model	<code>ollama pull llama3.2</code>

20.2 Glossary

Term	Meaning
AI Assistant	The integrated assistant panel used to help explain, review, generate, or troubleshoot code and project tasks.
Dependency	An external tool required for certain workflows, such as Git, Node.js, Python, Ollama, or Visual Studio C++ Build Tools.
MSI	Microsoft Installer package commonly used for Windows administration and managed deployment.
NSIS setup EXE	A standard Windows setup executable created with Nullsoft Scriptable Install System.
Ollama	A local AI model runtime used to run models on the user computer.
Workspace	The active project folder opened inside Diligent Code Studio.
Workspace Menu	The app navigation area used to reach editor, setup, settings, and other workspace functions.

20.3 Future Manual Improvements

- Add screenshots of each major v0.6.10 screen after final UI capture is available.
- Add exact log file locations after they are finalized in the application.
- Add Debian/Linux installation instructions when the Linux release artifact is finalized.
- Add a one-page quick-start handout for end users.
- Add administrator deployment notes for GitHub Releases, website downloads, and signed installers.

End of Manual

Diligent Code Studio v0.6.10